

```
public class GenericArrayList<T>
{
    private T[] Arr { get; set; }
    public GenericArrayList() { Arr = new T[10]; }
    public T Get(int i) { return Arr[i]; }
    public void Set(T t, int i) { Arr[i] = t; }

    public static void Test()
    {
        GenericArrayList<int> l = new GenericArrayList<int>();
        l.Set(10, 0);
        int x = l.Get(0);
    }
}

public class TypeErasedArrayList
{
    private object[] Arr { get; set; }
    public TypeErasedArrayList() { Arr = new object[10]; }

    public object Get(int i) { return Arr[i]; }
    public void Set(object t, int i) { Arr[i] = t; }

    public static void Test()
    {
        TypeErasedArrayList l = new TypeErasedArrayList();
        l.Set(/ Boxing from int32 to object / 10, 0);
        Int32 x = (Int32) l.Get(0);
    }
}
```

Since `Object` is the root of the class hierarchy, it is safe to substitute `Object` for all type parameters (constrained type parameters are substituted for their constraint). To maintain the illusion that the code is generic, casts are inserted automatically. However, the implicit boxing (e.g., with `int`) and casts incur a performance hit. Furthermore, as evidenced above, type erasure disallows Java generics from being used in tandem with primitive types because of this translation. That is, this translation forces you to write `ArrayList<Integer>` instead of `ArrayList<int>`.